Investigating Developers' Soft-Ethical Preferences in the Age of LLM-Based Coding Assistants

Anonymous authors

***Abstract*—LLM-based coding assistants increasingly resolve implementation choices that would otherwise be made by developers, potentially reducing their agency over such decisions. Although different implementations may satisfy the same requirements, they can have different ethical implications for the resulting software. As developers delegate code writing to LLMs, their soft-ethical preferences may be replaced by the model's implicit defaults. This paper investigates how developers perceive their agency over discretionary implementation choices, which soft-ethical aspects characterize these choices, and how developers' preferences can be elicited to support their agency when delegating code generation. We investigated developers' perspectives through a survey of 104 practitioners, refined six soft-ethical aspects with a multidisciplinary expert panel, and developed and evaluated a scenario-based instrument for eliciting developers' preferences. Our findings show that 62.5% of respondents report substantial discretion over implementation choices, 85.3% acknowledge that personal priorities or values influence their decisions, and 82.7% report delegating at least some choices to LLM-based assistants. We identified six soft-ethical aspects and operationalized them in a 25-item questionnaire to elicit developers' preferences. The elicitation instrument aims to make developers' soft-ethical preferences explicit and to provide a foundation for future research on preference-aware code generation and review, with the goal of preserving developers' ethical agency over implementation decisions delegated to LLMs.**

***Index Terms*—Soft Ethics; Large Language Models; Coding Agents; Software Engineering for Artificial Intelligence.**

I. Introduction

LLM-based coding assistants increasingly handle complex software implementation tasks, from generating and modifying code to inspecting project context [1]–[3]. As developers delegate more implementation work to these systems, their role shifts toward specifying tasks, prompting, and reviewing generated artifacts [4], [5]. However, delegating code generation also means delegating implementation decisions that developers would otherwise make themselves, potentially reducing their agency over the resulting software.

Requirements and constraints rarely determine every implementation detail, leaving developers to choose among equally valid alternatives [6]. Consider a registration form whose requirements specify collecting users' gender without prescribing the available options. One developer may offer multiple options, including non-binary identities and the possibility not to disclose one's gender, while another may provide only binary options. Both implementations satisfy the requirements, yet they differ in how inclusively they accommodate users' identities. Such decisions illustrate what Floridi calls *soft ethics*: choices made beyond legal and regulatory obligations (*hard ethics*) to make technology more socially preferable [7]. These discretionary choices can affect software fairness, inclusiveness, privacy, and transparency [8]–[10]. Developers may also differ in how they resolve them, reflecting individual priorities and values. We refer to these as *soft-ethical implementation preferences*: value-laden preferences over implementation alternatives left open by requirements and constraints.

When code generation is delegated to an LLM, these decisions do not disappear; instead, the model makes them based on its training data, alignment mechanisms, and provided context, potentially introducing implicit defaults and biases [11]–[16]. Without explicit guidance, implementations may reflect the model's defaults rather than developers' preferences. Although developers can review generated code, reviewing a decision is not equivalent to making it: their agency shifts from making implementation choices to guiding and evaluating those made by the assistant.

Making developers' soft-ethical preferences explicit could help preserve this agency by informing delegated decisions and making their underlying values open to scrutiny. Existing mechanisms, such as prompts and agent context files (e.g., `AGENTS.md` and `CLAUDE.md`), allow developers to specify project goals and coding conventions [17], while ethical prompt engineering supports explicit consideration of ethical trade-offs [18]. However, they do not systematically characterize or elicit individual developers' soft-ethical implementation preferences. Before such preferences can inform LLM-based code generation, we need to understand *whether* developers perceive discretion over ethically relevant decisions, *which* soft-ethical aspects these decisions involve, and *how* individual preferences can be elicited. To address this gap, we investigate the following research question:

Main Research Question

How do developers perceive discretion over ethics-relevant implementation choices, and how can their soft-ethical preferences be characterized and elicited?

We surveyed 104 software developers to investigate their perceived implementation discretion, soft-ethical considerations, and agency when using LLM-based coding assistants. We then identified soft-ethical aspects from the literature and refined them through a multidisciplinary focus group with six experts. Based on these findings, we developed a scenario-based instrument to elicit developer soft-ethical profiles, reviewed by experts and piloted with four developers. Characterizing and eliciting these preferences is a prerequisite for using them to guide LLM-based code generation, which

remains outside the scope of this study.

Our findings show that 62.5% of respondents perceive substantial discretion over implementation choices, 85.3% acknowledge that personal priorities or values influence their decisions, and 82.7% report delegating at least some choices to LLM-based assistants. Yet soft-ethical considerations rarely emerge spontaneously among their criteria for accepting generated code. We also identified six soft-ethical aspects—fairness, inclusiveness, privacy, transparency and explainability, traceability, and resource consumption—and operationalized them into a 25-item scenario-based elicitation instrument.

This paper makes four contributions: (1) empirical evidence on developers' discretion, soft-ethical considerations, and perceived agency in LLM-based code generation; (2) a conceptualization of soft-ethical implementation preferences and six aspects describing their manifestations in code and software behavior; (3) a scenario-based instrument for eliciting developers' soft-ethical profiles, reviewed by experts and piloted with developers; and (4) a publicly available replication package containing the study materials and instrument [19].

II. Background and Related Work

This section discusses how developers' individual preferences may influence implementation choices, the ethical dimensions relevant to software engineering, and the implications of delegating implementation decisions to LLM-based coding assistants.

a) Developers and Implementation Choices: Software development involves individual judgment and creativity, which can shape developers' implementation decisions [20]. Developers approach the same programming task differently, drawing on their experience, practices, and individual preferences. These differences leave recognizable traces in source code. As demonstrated by Caliskan-Islam et al. [21], programmers can be identified through stylistic features of their code, suggesting that individual coding practices can shape the resulting artifacts.

Individual differences are also relevant to implementation decisions that are not fully determined by requirements or external constraints. Liang et al. [6] investigated how developers make such decisions, showing that they navigate trade-offs among alternative implementations based on considerations such as performance, readability, maintainability, and security. Thus, even when working under the same requirements, developers may select different implementation alternatives according to their priorities and judgments. This discretionary space allows for individual preferences to influence not only how code is structured and written, but also the behavior of the resulting software.

b) Ethical Dimensions in Software Engineering: Research on human values in software engineering has examined how values emerge in development artifacts [10], relate to software properties and engineering decisions [8], and can be operationalized into engineering practices [9]. Ethical concerns have also been organized into dimensions and more specific aspects. For example, Xu et al. [22] investigated ethical sourcing in code generation, identifying dimensions such as equity, subject rights, accountability, intellectual property rights, and sustainability, encompassing aspects including fairness, privacy, transparency, licensing, working conditions, and energy consumption. Ethics labels provide structured descriptions of ethically relevant properties of digital systems [23], while De Ninno et al. [24] highlight the importance of accounting for stakeholders' understanding of these properties.

Not all ethical considerations pertain to decisions left to individual discretion. Some ethical concerns involve mandatory constraints, such as compliance with software licenses *hard ethics*, whereas others may also involve discretionary decisions, such as providing additional explanations to users or limiting unnecessary resource consumption *soft ethics*.

Beyond describing ethical properties of systems, prior work has explored the representation of individual ethical preferences. Ethical profiles can support digital interactions reflecting users' moral preferences [25], as well as ethical negotiation among autonomous systems acting on behalf of individuals [26]. Eliciting such preferences requires accounting for their context-dependent nature, as individual ethical preferences may manifest differently across situations. To this end, previous research has explored scenario-based questionnaires and formal models for representing individuals' soft-ethical preferences [25], [27]. While these approaches target individuals as users of digital systems, our work focuses on developers' preferences over implementation alternatives that manifest in source code.

c) Developer Agency in LLM-Based Code Generation: LLM-based coding assistants increasingly support activities beyond code completion, including the generation and modification of software artifacts [1], [2]. Their adoption can shift developers' creative contributions toward problem framing, specification, prompting, and evaluation of generated artifacts [4], raising questions about how to preserve human agency in AI-assisted software development [5].

At the same time, LLM-generated code may exhibit ethically relevant properties, including social biases [12]–[16]. Existing approaches address ethical concerns through safeguards [28] and ethical prompt engineering, which emphasizes human oversight and explicit consideration of ethical trade-offs [18]. Developers can also steer generation through prompts and agent context files that specify project instructions and coding conventions [17].

However, none of these mechanisms provide a systematic way to characterize and elicit developers' individual preferences over discretionary, ethics-relevant implementation choices, which is the gap this paper addresses.

III. Empirical Study Design

The goal of the study is to characterize the discretionary space in which developers make soft-ethical implementation choices, to understand how to elicit developers' soft-ethical preferences in a form that can later guide LLM-based code

generation. The perspective is of both researchers and practitioners. To this end, we formulated three research questions:

> **RQ_1 (Developer's Agency)**. *To what extent do developers perceive discretion over implementation choices, consider soft-ethical aspects within them, and retain agency over them when using LLM-based coding assistants?*

> **RQ_2 (Soft-ethical Aspects)**. *Which soft-ethical aspects are relevant to developers' implementation choices, and how can they manifest in source code?*

> **RQ_3 (Ethical Profile Elicitation Instrument)**. *How can developers' soft-ethical preferences be elicited to inform the construction of developer ethical profiles?*

RQ_1 investigates developers' perceived discretion over implementation choices, the extent to which soft-ethical considerations influence choices, and how developers retain agency when using LLM-based coding assistants. RQ_2 focuses on identifying, validating, and refining the soft-ethical aspects relevant to such choices, as well as discussing how these aspects may manifest in source code. Together, RQ_1 and RQ_2 provide the empirical and conceptual foundations for RQ_3. Building on their results, RQ_3 develops an instrument to elicit developers' soft-ethical preferences.

We addressed the research questions through a mixed-method study combining quantitative and qualitative approaches. Figure 1 overviews the study design. For RQ_1, we conducted a survey with software developers to investigate their perceived discretion over implementation choices and collect practical evidence of ethics-relevant considerations (Section III-A). For RQ_2, we identified an initial set of soft-ethical aspects from the literature and refined and validated them through a multidisciplinary focus group (Section III-B). The focus group also provided guidance for designing the ethical profile elicitation instrument. Building on this guidance, for RQ_3 we developed an elicitation instrument, in the form of a scenario-based questionnaire, informed by the refined aspects, the focus group's recommendations, and the practical examples collected through the survey (Section III-C). Eventually, the instrument has been assessed through semi-structured interviews with multidisciplinary experts and software developers. The first two research questions were not answered in the order of their presentation: we ran the focus group for RQ_2 first, and the survey for RQ_1 incorporated aspects refined in the focus group. We present RQ_1 first because it establishes the relevance of the problem on which the study builds.

A. RQ_1 — Developer's Agency

To address RQ_1, we conducted an exploratory cross-sectional survey with software developers, following the ACM SIGSOFT Empirical Standard for questionnaire surveys [29]. It investigated (i) the implementation discretion perceived by developers, i.e., the decisions not determined by requirements or by organizational, legal, and technical constraints; (ii) whether soft-ethical considerations influence these decisions; and (iii) developers' perceived agency when using LLM-based coding assistants. As we aimed to characterize the phenomenon rather than estimate its prevalence, we did not seek statistical generalization [30].

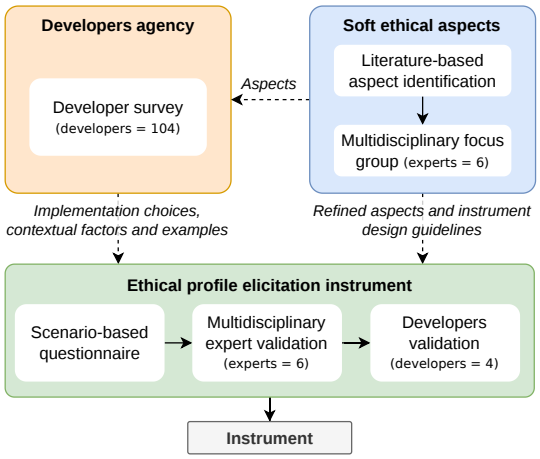


Fig. 1. Overview of the study design.

Survey design. After introducing the study goals, data handling, and informed consent, the questionnaire comprised four sections. The first collected the respondent's development context, i.e., role, years of experience, organization type, location, application domain, and target users. The second asked which aspects of a typical task are specified before implementation (e.g., architecture, data to be collected, accessibility requirements), delimiting the perimeter of developer discretion. The third addressed LLM-based coding assistants: frequency of use, supported activities, how often generated code is accepted without substantial modification, whether functional correctness alone suffices to accept it, and how much the assistant makes choices respondents would otherwise make. The fourth focused on the decisions left open once all constraints are defined: how much discretion respondents usually have, which choices are left to them (e.g., default values, logging details, information shown to users), and whether personal priorities or values influence the choice among equivalent implementations. Respondents then rated how often they consciously consider each soft-ethical aspect (see Table I) on a 5-point frequency scale, each presented with a definition and an example, and were asked to (i) describe a recent choice of this kind, (ii) state whether they would make it in a different context and which domain, purpose, or user factors would matter, and (iii) report whether they feel they delegate such choices to the LLM. A final open question collected further considerations on their discretionary space. The questionnaire, anonymized responses, and process details are in our publicly available replication package [19].

Pilot. Four software engineering experts piloted the survey: two practitioners and two researchers, each with at least five years of industrial or academic experience and expertise in empirical studies and questionnaire design. Completion took

9–11 minutes. Based on their feedback, we (i) specified that frequency questions refer to an average task; (ii) added a question on whether functional correctness suffices to accept generated code; (iii) reworded the question on personal values; and (iv) simplified questions perceived as too complex. Pilot responses were excluded from the analysis.

Participants. We used purposive sampling [30], targeting developers currently working in industry, as they make implementation decisions in real projects under actual constraints; respondents could also hold research roles, since developers may combine both activities. We recruited through three channels, i.e., Prolific,[1] direct invitations, and developer communities on Reddit, all leading to the same Google Form. Prolific is increasingly recognized in software engineering research as a viable platform for recruiting practitioners [31], [32], provided that rigorous filtering and screening are applied. Despite persisting concerns about online participants, prior studies show that it provides high-quality respondent pools when appropriate safeguards are in place [33], [34]. Accordingly, we used Prolific's screening filters to restrict participation to people currently employed in software engineering or development, experienced in programming, and with knowledge of programming languages. Prolific participants received $2.50, whereas participation through the other channels was voluntary and uncompensated. Overall, we collected 104 responses (75 via Prolific, 25 via direct invitations, and 4 via Reddit), excluding the 4 pilot participants, and anonymized them before the analysis.

Analysis. We analyzed closed-ended questions through descriptive statistics, i.e., the proportion of respondents selecting each option and the distributions of frequency and agreement ratings, without inferential tests, consistently with the exploratory goal of the survey. To understand the considerations behind developers' choices, we complemented them with a qualitative content analysis of open-ended responses [35]. We coded the descriptions of recent discretionary choices deductively, using the six soft-ethical aspects as categories, distinguishing choices whose ethical relevance respondents explicitly acknowledged from those whose implementation or motivation could be interpreted as reflecting these aspects, and also considering choices primarily motivated by technical preferences or established coding practices. We inductively categorized the contextual factors influencing developers' choices, grouped the criteria for accepting LLM-generated code into recurring categories, and examined additional free-text comments to contextualize the quantitative findings. One author conducted the analysis and a second reviewed the classifications and interpretations, resolving disagreements through discussion. Besides answering RQ_1, the qualitative findings provided concrete implementation examples that informed the refinement of the aspects in RQ_2 and the development of the scenario-based items in RQ_3.

[1]Prolific: https://www.prolific.com

B. RQ_2 — Soft-Ethical Aspects

To address RQ_2, we identified seed ethical aspects from the literature and then validated and refined them through a multidisciplinary expert focus group.

Aspect identification. Rather than building an exhaustive taxonomy of ethical concerns from scratch, we drew on existing literature to identify *seed ethical aspects* potentially relevant to developers' discretionary implementation choices. We started from the ethics label by Autili et al. [23], which relates properties of digital systems to ethical concerns, and from Xu et al. [22], who investigate ethical aspects of code generation, complemented by research on human values in software engineering [8], [9] for a broader perspective on the values that may influence developers' decisions.

We selected aspects that (i) could plausibly reflect preferences varying across developers; (ii) involve implementation choices that could affect end users (hereafter *user-facing*) or other developers (*developer-facing*); and (iii) could manifest in source code or in the behavior it encodes. Consistently with our focus on soft ethics [7], we considered discretionary choices beyond those fully determined by functional or non-functional requirements, laws, regulations, or applicable standards.

One author identified the initial seed aspects and drafted their descriptions; a second author reviewed them against the selection criteria, the literature on human values in software engineering, and their potential implementation manifestations; and a third author contributed to refining the candidates. This led to an agreed set of five seed aspects: *fairness*, *inclusiveness*, *transparency and explainability*, *traceability*, and *energy consumption*. We adopted the naming of Xu et al. [22], as their work specifically addresses code generation rather than software systems broadly, but not exclusively their definitions: we compared how the selected sources characterized each aspect and integrated the overlapping and complementary elements relevant to developers' discretionary choices. For each aspect, we then formulated a working description and representative manifestations in software artifacts, drawing on the selected literature and our software engineering domain knowledge.

Focus group design. To review and refine the seed aspects and inform the design of the elicitation instrument, we conducted a focus group, a method that explores participants' perspectives through moderated group discussion [36]. Its objectives were (i) assessing the conceptual adequacy, coverage, and distinctiveness of the aspects; and (ii) exploring how developers' preferences regarding them could be elicited, to inform the instrument of RQ_3. Through direct email invitations, we recruited a purposive, multidisciplinary panel of six participants: an industry AI ethicist, an industry developer, and researchers in law, anthropology, software engineering, and digital humanities. They were selected for their complementary expertise in the ethical, legal, and social concerns of computational systems, with professional or research experience ranging from over three to more than ten

years. This heterogeneous composition reflects the normative, legal, social, and ethical nature of soft-ethical concerns in code, while the developer grounded the discussion in practical knowledge of implementation decisions. The session was held synchronously online, lasted approximately 90 minutes, was moderated by two authors, and was recorded and transcribed with participants' informed consent.

Session structure. The session comprised two parts, one per objective. In the first, participants were presented with the five seed aspects, with their working descriptions, potential manifestations in code, and representative implementation examples. They discussed whether the aspects captured meaningful ethical considerations in software development, whether their definitions were clear and sufficiently distinct, whether relevant concerns were missing or required refinement, and whether the manifestations represented decisions genuinely available to developers rather than predetermined by functional, organizational, or legal constraints. This feedback informed the refinement of the seed aspects. The second part explored how developers' soft-ethical preferences could be elicited, providing guidance for developing the instrument described in RQ_3.

Analysis. The two moderators produced a condensed transcript of the recording, removing repetitions and side exchanges while preserving the substantive content, retaining speaker identifiers, and distinguishing moderator from participant contributions. They then synthesized the emerging insights into actionable implications concerning (i) the conceptualization of the aspects and (ii) the design of the elicitation instrument, which the authors discussed to determine whether and how each should inform the study. Accordingly, two authors revised the seed aspects by adding missing aspects, rephrasing unclear or ambiguous descriptions, and improving the examples (Section V). The outputs of this step are the revised aspects in Table I and the design requirements for the instrument, used to answer RQ_3.

C. RQ_3 — Ethical Profile Elicitation Instrument

To address RQ_3, we operationalized the aspects from RQ_2 into a self-report instrument, building on the developers' perspective gathered in RQ_1, and investigated how the resulting profile should be represented. Following the staged process outlined by Graziotin et al. [37], we developed it in four steps, i.e., instrument design guidance, item generation, expert review, and piloting with developers, a process also adopted in software engineering for self-assessment instruments [38]. These activities focused on assessing and refining the instrument's relevance, clarity, and plausibility. Establishing its psychometric properties would require further validation, which remains outside the scope of this study.

Instrument design guidance. Inspired by prior questionnaire-based approaches to eliciting individuals' soft-ethical preferences [25], we explored during the focus group how to elicit developers' preferences. Participants were presented with preliminary statements illustrating implementation choices associated with the seed aspects, and the discussion, moderated by two authors, focused on whether these statements represented realistic implementation decisions, whether they could serve as questionnaire items, and how the response format should be structured. It yielded design considerations on the content, structure, and wording of the items and on the response format, which informed the development of the questionnaire.

Item generation. Two authors formulated items describing implementation scenarios that could fall within developers' discretion, drawing on the aspects refined in RQ_2, the manifestations discussed in the focus group, and, where applicable, the practical examples collected in RQ_1, following the guidelines by Kitchenham and Pfleeger [39]; all authors reviewed them. This resulted in a questionnaire of 25 items, four per aspect (five for *transparency and explainability*).

Expert review. To assess the content and formulation of the items, we involved six experts with backgrounds in ethics and software engineering, recruited through direct email invitations. We first recontacted the focus group participants: four agreed, and two replacements maintained a comparable multidisciplinary composition, i.e., a computer science researcher with software development expertise replaced the industry developer, and a philosopher specialized in trustworthy AI replaced the industry AI ethicist. In asynchronous semi-structured interviews, experts reviewed the description and the scenario-based items of each aspect, assessing whether the scenarios (i) involved ethically relevant considerations, (ii) primarily reflected the intended aspect, (iii) presented meaningful differences between implementation alternatives, and (iv) could provide evidence of developers' preferences, as well as the suitability of the response scale. One author collected, transcribed, and summarized the responses, which the authors then discussed to determine whether and how to refine the items and completion instructions, revising the questionnaire where appropriate. Where necessary, follow-up exchanges with experts clarified concerns about developer-facing choices, particularly those raised by experts without development experience.

Piloting with developers. We then piloted the refined elicitation instrument with four industry developers, each with more than six years of professional experience, recruited through direct email invitations among the RQ_1 respondents willing to participate in follow-up studies. Participants independently completed the questionnaire, with items in randomized order to mitigate order effects, and then took part in asynchronous semi-structured interviews assessing (i) the clarity of the scenario descriptions, (ii) the plausibility of the proposed implementation choices, and (iii) the applicability of the scenarios across development contexts. Participants were also asked whether the scenarios captured decisions that could reflect their individual soft-ethical preferences, and whether they felt pressured to provide socially desirable rather than genuine responses. Two authors reviewed the feedback to identify issues of comprehension, relevance, and interpretation, with particular attention to ambiguities, unrealistic choices, perceived implementation effort, and contextual factors affect-

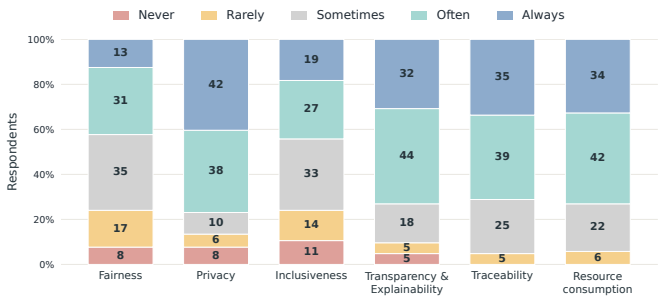


Fig. 2. Respondents' considered soft-ethical aspects.

ing plausibility. Based on this review, they refined the wording, contextualization, and instructions of the questionnaire where appropriate, resulting in the final version of the instrument.

IV. Developer's Agency Consideration (RQ_1)

The survey collected 104 responses from software practitioners with different professional backgrounds and development contexts. Most were software developers or engineers (53.8%) or senior developers (27.9%), with 83.7% reporting at least three years of professional experience. Participants worked primarily in private companies (76%), across 23 countries and diverse development areas and application domains, particularly backend and full-stack development (51.9% each), finance, and developer tools (26-28%). Drawing on this heterogeneous sample, we investigated developers' perceived discretion over implementation choices, the extent to which these choices reflect soft-ethical considerations, and how developers perceive and exercise agency when using LLM-based coding assistants.

Regarding perceived discretion on implementation choices, the majority of respondents (62.5%) acknowledged that *many* or *most* implementation choices remain open after requirements and project constraints have been defined, while 27.9% reported *some* discretion and only 9.6% reported *few* remaining choices. This discretion spans different types of implementation decisions, particularly error handling, input validation, code readability and maintainability, data structures, and algorithms (63.5-70.2% of respondents each). Importantly, respondents also reported discretion over choices potentially affecting soft-ethical aspects, including logging details and resource usage (49% each), as well as accessibility-related implementation details, information shown to users, and privacy-related details (30.8-34.6%). Nevertheless, respondents' explanations suggest that such discretion depends on the application domain, since domain-specific regulations, security standards, organizational policies, and architectural constraints may restrict the range of available implementation choices.

> **RQ_1 — Answer#1:** Developers report substantial discretion over implementation choices affecting both code structure and observable software behavior, including choices with potential soft-ethical implications.

Regarding the considerations shaping discretionary implementation choices, 85.3% of respondents acknowledged that their personal priorities or values may influence their decisions. While 42.3% were already aware of this influence, 43.3% recognized it only when prompted to reflect on it. Moreover, 65.4% reported that they *always* or *often* reflect on the broader consequences of their implementation choices, whereas 13.5% acknowledged that they do it depending on the application or development context. Among the six soft-ethical aspects, *Privacy* was the most frequently considered by developers when making implementation choices (76.9% often or always considered), followed by *Transparency and Explainability*, *Resource Consumption*, and *Traceability*. *Fairness* and *Inclusiveness* were considered less frequently. Figure 2 summarizes the obtained responses on the considerations of ethical aspects. Respondents also differed in their relative priorities: the 104 respondents produced 92 distinct rating patterns across the six aspects, only 9 (8.7%) rated all aspects equally, and 38 of the remaining 95 (40%) rated at least one other aspect above *Privacy*.

The qualitative analysis of responses to the open-ended question asking developers to describe a recent implementation choice left open by requirements and project constraints provides concrete examples of how personal priorities and soft-ethical considerations can shape such decisions. Some respondents explicitly acknowledged such considerations in their decisions. For instance, one respondent (`p_039`) described voluntarily implementing error handling mechanisms that balance privacy and traceability:

> *"I decided on my own not to include any sensitive data [...], nor to log any personal data that the user had entered incorrectly – only the type of error and an anonymized identifier. [...] No one asked me to do this, but I did it with privacy, security, and traceability in mind."*

Other respondents described choices with soft-ethical implications without explicitly framing them as ethical considerations, instead emphasizing technical or practical benefits. These choices concern both developer-facing properties and user-facing behavior. For example, one respondent (`i_002`) reported introducing detailed logging into a CI/CD pipeline *"to help developers investigate errors or failed releases"*, hence reflecting developer-facing traceability motivated by debugging needs. In contrast, another respondent (`p_033`) described choosing to *"display clear error messages to the user instead of generic ones"*, illustrating how transparency and explainability can inform user-facing implementation choices. Further examples illustrate how discretionary choices may concern other soft-ethical aspects. One respondent (`p_047`) reported choosing to return *"a confidence score and an explanation to the user instead of just a binary label"* when implementing AI classification, while another (`p_036`) described having discretion over *"what data to ask and what not to ask, as well as how accessible the page should be to different people"*, highlighting choices affecting transparency, privacy and inclusiveness.

70 of the 104 respondents (67.3%) indicated they might

make different implementation choices depending on the application context. The qualitative analysis of their explanations identified several influencing factors, such as the application purpose, the sensitivity of the handled data, the characteristics of the users, and performance or resource constraints. For example, one respondent (p_055) described prioritizing privacy, security, and traceability in banking applications handling sensitive customer data, while emphasizing accessibility and clear error messages in customer-facing applications. Similarly, another respondent (p_031) explained that user expertise, vulnerability, and application criticality influence the importance of explainability, fairness, and traceability, stating that they would *"write a developer tool and a consumer health app very differently, even with the same requirements on paper"*. Other respondents described how specific user characteristics influence implementation decisions, such as adapting error messages to users' technical expertise (p_004) or accommodating screen-reader and accessibility needs (r_002). Finally, some respondents noted that regulatory and organizational constraints may further restrict the available discretionary choices, rather than merely influencing their priorities.

> **RQ_1 — Answer#2:** Developers report considering personal priorities and soft-ethical aspects when making discretionary implementation choices. While some developers explicitly recognize the ethical relevance of their decisions, others may make choices that reflect such aspects without explicitly recognizing their ethical implications. Such decisions are often context-dependent, with application domain and intended users influencing developers' priorities.

LLM-based coding assistants were widely used among respondents, with 74% reporting frequent or very frequent use. Beyond code generation (79.6%) and debugging (83.5%), respondents used them to explain or understand the code (62.1%) and to explore alternative implementations (54.4%). Furthermore, 43.3% perceived that assistants substantially or very strongly make implementation choices they would otherwise make themselves, while 86 developers (82.7%) explicitly acknowledged delegating at least some choices. To retain control over these decisions, 30 respondents (28.8%) reported guiding assistants through prompts and 48 (46.2%) reported reviewing and modifying generated code. Nevertheless, 41.3% reported often or very often accepting generated implementations without substantial modification.

When evaluating generated implementations, 68 out of 104 respondents (65.4%) reported considering criteria beyond functional correctness, primarily coding standards and conventions, performance, maintainability, security, readability, and architectural consistency, each mentioned by approximately 18–27% of respondents. These criteria were predominantly framed in terms of technical quality and compliance with existing project constraints. Some overlapped with soft-ethical aspects, particularly code readability, which contributes to developer-facing transparency. For instance, one respondent (i_007) mentioned *"code readability, maintainability, and size of diffs to be reviewed by peers"*. Privacy was mentioned by only two respondents, but only in relation to organizational or compliance requirements, rather than as a discretionary implementation choice. No respondents explicitly mentioned fairness, inclusiveness, or user-facing explanations of system behavior among their acceptance criteria. Similarly, although performance and efficiency were frequently mentioned, respondents did not explicitly frame these concerns in terms of minimizing resource consumption. Interestingly, this contrasts with the discretionary implementation choices reported by developers, which included concrete examples of soft ethical aspects consideration (as summarized in Answer#2). In some cases, the difference emerged even within the same respondent's answers. For example, one participant (p_039) described autonomously deciding not to log personal data to protect privacy, while another (p_047) reported providing explanations alongside AI-generated classifications. However, both indicated that functional correctness was sufficient to accept the generated code. This contrast should not be read as a contradiction. Acceptance criteria were collected before the six aspects were defined and illustrated, whereas implementation choices were described afterwards. Rather, it suggests that soft-ethical considerations, particularly those affecting end users, have low spontaneous salience: developers rarely bring them up unprompted when evaluating generated code, yet recognize them in their own choices when asked to reflect on them.

> **RQ_1 — Answer#3:**: Developers report delegating discretionary implementation choices to LLM-based coding assistants while seeking to retain agency through prompting and code review. When stating their acceptance criteria unprompted, they emphasize functional correctness, technical quality, and project constraints, while soft-ethical considerations, particularly those affecting end users, rarely emerge, even though developers recognize them in their own implementation choices once asked to reflect. This motivates investigating elicitation means for making developers' preferences available to inform delegated implementation decisions.

V. Soft-ethical aspects (RQ_2)

The literature-informed identification process resulted in five seed ethical aspects: *fairness*, *inclusiveness*, *transparency and explainability*, *traceability*, and *energy consumption*. These aspects, together with their working descriptions and representative implementation manifestations, were presented to the multidisciplinary expert focus group for review and refinement. Experts generally considered the proposed aspects relevant to software development and their descriptions sufficiently clear and distinct, while identifying opportunities to improve their conceptualization and coverage.

A first outcome concerned the scope of the identified aspects. In particular, an expert highlighted that *privacy* was missing as an independent concern. Privacy-related considera-

tions, such as the collection and processing of personal information, had initially been included within fairness. However, the discussion highlighted that these considerations warranted separate treatment, leading us to introduce *privacy* as a sixth aspect. Consistent with our focus on soft ethics, this aspect captures developers' discretionary choices concerning personal data that go beyond what is prescribed by legal and regulatory obligations, rather than privacy requirements mandated by such obligations. Similarly, another expert suggested broadening *energy consumption* to *resource consumption*, since implementation choices may affect computational, network, storage, and other resources beyond energy alone. We adopted this broader terminology in the revised set.

The discussion also informed the formulation of the aspects description. Experts emphasized that the descriptions should consistently reflect developers' perspectives, focusing on preferences among implementation alternatives rather than solely on properties of the resulting software. For instance, the initial definition of *fairness* was revised to separate privacy-related considerations and encompass both avoiding unjustified distinctions and actively supporting equitable treatment, as illustrated below.

> ***Fairness (seed):*** *Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.*
>
> ***Fairness (refined):*** *Developers may prefer implementation choices that support equitable treatment across users, avoid encoding stereotypes, and differentiate users only when such distinctions are functionally relevant and justified.*

Experts further highlighted the importance of concrete examples for illustrating how abstract ethical concerns may manifest in code. Accordingly, we revised the descriptions to consistently reflect developers' perspectives and complemented each aspect with representative examples of implementation manifestations. The exemplified manifestations cover both (i) the static representation of code, including implementation structures, identifiers, comments, and dependencies (*developer-facing*), and (ii) the behavior encoded by code, including user interactions, data processing, logging, and resource consumption (*user-facing*).

These refinements resulted in the six soft-ethical aspects presented in Table I: *fairness*, *inclusiveness*, *privacy*, *transparency and explainability*, *traceability*, and *resource consumption*. For each aspect, the table reports its description in terms of developer preferences and the illustrative implementation manifestations. These aspects do not represent mutually exclusive properties or replace mandatory software requirements. Rather, they provide a conceptual basis for investigating how developers prioritize ethics-relevant considerations when multiple valid implementation alternatives are available.

> **RQ_2 — Answer:** We identified and described six soft-ethical aspects relevant to developers' discretionary implementation choices: *fairness*, *inclusiveness*, *privacy*, *transparency and explainability*, *traceability*, and *resource consumption*. These aspects may manifest both in the static representation of source code and in the behavior it encodes.

VI. Ethical Profile Elicitation Instrument (RQ_3)

The design of the instrument was informed by the findings of RQ_1 and RQ_2 and insights that emerged from the multidisciplinary focus group. Both the survey and the focus group highlighted that developers' soft-ethical preferences may depend on the development context. Participants suggested using concrete, actionable implementation scenarios, presented in neutral terms, to elicit the choices developers would make, without framing any particular responses as inherently more ethical, to avoid encouraging socially desirable responses. The discussion also highlighted how the presentation of implementation trade-offs may influence developers' responses, and that some choices may be constrained by external requirements. Concerning the answer type, participants strongly advised proposing qualitative options instead of asking for a numerical quantification of preferences.

Following these insights, we developed a scenario-based questionnaire to elicit developers' soft-ethical preferences through hypothetical implementation choices. The questionnaire comprises 25 items, each associated with one of the identified soft-ethical aspects and presenting a choice between two implementation alternatives that satisfy the same functional requirements, but differ in their ethical implication. The scenarios cover both developer-facing and user-facing manifestations of the aspects, drawing on real examples and considerations collected through the RQ_1 survey, where applicable. In total, for each aspect, we defined four scenarios, except for *transparency and explainability*, which has five. To account for the context-dependent nature of soft-ethical preferences, we avoided prescribing a specific application context within the scenarios where possible. Instead, respondents are instructed to consider a particular development context when completing the questionnaire and use it as a reference for evaluating all scenarios.

Each scenario-based item is structured in three parts: (i) a brief description of an implementation task, (ii) two alternative implementation solutions differing in the ethical implication but both satisfying the system requirements, and (iii) a statement expressing a preference for one of the alternatives. Developers are asked to indicate how likely they would be to make the stated choice using a qualitative five-point Likert-type scale, ranging from *very unlikely* to *very likely*, including a central neutral option. To reduce the tendency to consistently favor one response direction, the instrument includes both positively and reverse-keyed items. In positively keyed items, the statement expresses a preference for the implementation solution that is more aligned with the ethical aspect; higher likelihood ratings indicate stronger alignment with the corresponding aspect. In reverse-keyed items, the relationship is inverted. For instance, a reverse-keyed item falling within the inclusiveness aspect is formulated as follows:

TABLE I
CONSIDERED DEVELOPER ETHICAL ASPECTS

Aspect	Description	Implementation Manifestations
Fairness	Developers may prefer implementation choices that support equitable treatment across users, avoid encoding stereotypes, and differentiate users only when such distinctions are functionally relevant and justified.	Code can embed stereotypes or assumptions that may result in biased or differential treatment, even when the implementation is functionally correct. For example, identifiers and comments may associate stereotypical roles, behaviors, or capabilities with demographic groups, while an implementation may apply different validation, eligibility, or processing rules to users based on characteristics that are not functionally relevant.
Inclusiveness	Developers may prefer implementation choices that accommodate users with different abilities, languages, devices, cultural conventions, gender identities, and needs, and that avoid implicitly centering only a narrow group of users.	Code can encode assumptions about users' abilities, interaction modes, linguistic and cultural conventions, gender identities, or locally specific data formats. For example, a form may restrict gender to binary options, whereas an implementation may fail to accommodate users with different accessibility needs and accept input formats that follow different national or cultural conventions.
Privacy	Developers may prefer implementation choices that minimize the collection, processing, storage, exposure, or sharing of personal data beyond what is functionally necessary for the system.	Code can collect, process, store, log, transmit, or expose user data beyond what is necessary to provide the intended functionality. For example, a form may request unnecessary demographic attributes, an application may use information that is not needed for its operation, or data may be retained, transmitted, or exposed beyond what the task requires.
Transparency and Explainability	Developers may prefer implementation choices that make system behavior understandable to both maintainers and end users, through readable and self-explanatory implementation logic as well as clear user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what actions the system takes. For example, developer-to-developer understanding can be supported through clear code structure, self-explanatory identifiers, and meaningful comments, while developer-to-user understanding can be supported through explanations of why information is requested, clear labels, and informative feedback about system actions and outcomes.
Traceability	Developers may prefer implementation choices that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. For example, traceability can be supported through explicit operation or submission states, meaningful error reporting, traceable validation or decision rules, audit-oriented events or logs, and mechanisms that allow users or maintainers to review relevant actions and outcomes.
Resource Consumption	Developers may prefer implementation choices that minimize unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while consuming more resources than necessary. For example, an implementation may rely on unnecessarily heavy dependencies, perform redundant computations, trigger excessive client-side updates, make repeated or unnecessary network requests, or introduce avoidable storage and runtime overhead.

> A form asks users to enter their name. It can accept Latin characters only, therefore users whose names use other writing systems can enter a transliterated version, or it can support names written in their original scripts, which requires broader input handling and validation. Both implementations allow users to provide their names. *I would keep the support for Latin characters only.*

A numerical score is associated with each of the response options. For positive-keyed items, the score ranges from 1 for *very unlikely* to 5 for *very likely*. For reverse-keyed items, the range is inverted, so that higher values consistently indicate greater alignment with the item's corresponding aspect. Respondents are not shown the numerical values assigned to the response options, nor are they informed of the soft-ethical aspects associated with each item.

The expert review generally supported the relevance of the scenario-based items and the suitability of the response scale, while identifying several concerns. In particular, experts questioned whether some scenarios adequately captured the intended ethical aspect, whether certain choices genuinely fell within developers' discretion, and whether responses could be interpreted independently of contextual factors. We observed differences in how experts interpreted the scenarios depending on their disciplinary backgrounds, with non-technical experts less frequently recognizing the ethical relevance of developer-facing implementation choices. We accounted for these differences when interpreting the feedback. Assessing whether developer-facing choices are realistic and discretionary requires technical expertise. Therefore, we retained scenarios grounded in examples of implementation choices identified through the RQ_1 survey and supported by software engineering experts, while addressing the concerns raised by other experts through clearer formulations. Other feedback was used to refine the scenarios and improve their clarity and formulation.

Following the expert review, we piloted the instrument with four software developers, who completed the questionnaire and provided feedback through semi-structured interviews. Developers generally found the scenarios understandable and plausible, although some noted that additional contextual information would help assess certain choices, particularly those with architectural implications. All participants agreed that the proposed implementation choices could reflect developers' individual preferences concerning the considered soft-ethical aspects. None reported feeling pressured to provide socially desirable responses rather than expressing their genuine preferences. Further discussion highlighted three additional considerations: (i) scenarios' stated choice too often is the one describing the implementation option more aligned with the related ethical aspect; (ii) some implementation choices may already be constrained by business or system requirements; (iii) the ethical significance of the choices may vary across scenarios. To address the first concern, we balanced the occurrence of positively and reverse-keyed items in the final instrument. Regarding the second concern, we retained the scenarios because, even when implementation choices are constrained by requirements in real projects, selecting among hypothetical alternatives can still reveal developers' personal preferences. For the last concern, we deliberately avoided prescribing a specific application context in the scenarios where possible, allowing respondents to evaluate the choices consistently with the development context identified

while completing the questionnaire. We acknowledge that the instrument captures self-reported preferences in hypothetical contexts rather than actual implementation decisions, and does not account for differences in the relative ethical significance of the scenarios.

Proof of concept. To illustrate how responses to the instrument can inform the construction of potential developer soft-ethical profiles, we developed a browser-based proof of concept that administers the scenario-based questionnaire in randomized order and aggregates responses into six aspect-level scores, according to the scoring scale described above. For each aspect, the score is computed as the mean of its item scores. The resulting six-dimensional representation is visualized through a radar chart, and the prototype supports exporting both item-level responses and aggregated aspect-level scores. To illustrate the profile construction procedure , we applied it to the questionnaires completed by the four developers during the pilot, obtaining four preliminary descriptive profiles. These preliminary profiles summarize developers' self-reported tendencies across hypothetical implementation choices, interpreted with reference to a specific development context. They should therefore be understood as potential, context-dependent representations of developers' soft-ethical preferences, rather than definitive or context-independent characterizations. The instrument, its prototype and anonymized pilot profiles are available in the replication package [19].

> **RQ_3 — Answer:** We developed a scenario-based questionnaire comprising 25 items to elicit developers' soft-ethical preferences through hypothetical discretionary implementation choices, reviewed by multidisciplinary experts and piloted with developers. The instrument supports eliciting self-reported preferences with reference to a specific development context, providing a basis for constructing potential, context-dependent developer ethical profiles.

VII. Threats to Validity

External validity. RQ1 relies on developers' self-reported perceptions of their discretion over implementation choices. Although these perceptions provide insights into how developers experience their decision-making autonomy, they may not accurately reflect the discretion actually available to them in their development contexts, which was not independently observed or assessed.

Internal validity. The survey for RQ1 collected developers' acceptance criteria for LLM-generated code before introducing the soft-ethical aspects, whereas examples of discretionary implementation choices were collected after these aspects had been presented. Therefore, the latter responses may have been influenced by the preceding questions, and differences between the two should be interpreted cautiously. Moreover, four experts participated in both the focus group and the subsequent instrument review, potentially introducing confirmation bias. Their continued involvement was motivated by their familiarity with the study's concepts and objectives, which enabled more informed feedback, but may also have influenced their evaluations. This threat was mitigated by involving additional experts in the instrument review.

Construct validity. Although expert review supported the relevance of the instrument scenarios to their intended soft-ethical aspects, developers may interpret them differently, and individual scenarios may involve multiple aspects. We mitigated this threat by including multiple items per aspect, hence reducing influence of potential ambiguities on the resulting profile. Furthermore, scenarios may implicitly suggest that one implementation alternative is more ethically desirable, potentially influencing responses. Although developers in the pilot did not report feeling pressured to provide socially desirable answers, we mitigated this threat by balancing positively and reverse-keyed statements. It's also worth to note that the aggregation of questionnaire responses into six-dimensional profiles proposed for the proof of concept, based on the mean item scores for each aspect, has not been empirically validated. Consequently, the resulting scores should be regarded as preliminary descriptive indicators rather than validated quantitative measures of developers' soft-ethical preferences. Furthermore, whether such profiles can effectively inform LLM-based code generation remains to be investigated. In general, the whole elicitation instrument was qualitatively reviewed for content relevance, clarity, and plausibility, but its actual psychometric properties still require assessment. The four-developer pilot was designed to identify problems in item interpretation, not to estimate scores reliability or validity. Consequently, the preliminary profiles developed in the proof of concept implementation should not be interpreted as validated measures of ethical preferences or as predictors of actual implementation behavior.

VIII. Discussion

Our findings highlight developers' discretion over implementation choices with soft-ethical implications, while raising questions about how their preferences can remain explicit when code generation is delegated to LLM-based assistants. We discuss three potential directions of our work.

Guiding LLM-based code generation. A primary direction is using developers' soft-ethical profiles to preserve their agency when delegating code generation to LLM-based assistants. When requirements leave multiple valid alternatives, an elicited profile could guide generation based on the developer's expressed preferences, rather than relying on the model's implicit defaults. The goal is not to prioritize individual preferences over established obligations or team policies, but to make otherwise implicit implementation choices explicit and open to scrutiny, including potential divergences between developers' preferences and those reflected in generated code. Since preferences may depend on context, developers should retain the ability to review and reconsider profile-guided decisions.

Supporting code review. A second direction concerns using elicited preferences to support the review of generated implementations. Since our findings suggest that developers

do not always spontaneously consider soft-ethical implications when evaluating LLM-generated code, review mechanisms could highlight potential divergences between generated implementations and developers' profiles. Rather than treating these divergences as ethical violations, such mechanisms could prompt reflection and informed oversight, potentially extending to conventional peer code review.

Supporting empirical studies of developer choices. A third direction concerns using the proposed instrument to investigate developers' soft-ethical decision-making. Once further validated, it could support studies examining how preferences vary across contexts and relate to actual implementation decisions. Comparing elicited preferences with observed choices could also help investigate whether delegation to LLM-based assistants affects the extent to which developers' preferences are reflected in code.

Broader societal impacts. Discretionary implementation choices can shape how software treats and accommodates users, affecting fairness, accessibility, cultural and linguistic inclusion, privacy, and transparency. Making developers' preferences explicit could help preserve their influence over these choices when delegating code generation to LLM-based assistants, potentially contributing to more socially responsible software development. While such profiles might also be considered for organizational purposes, such as team composition or task assignment, these applications would require additional methodological and ethical considerations. In particular, self-reported preferences should not be interpreted as measures of developers' ethical qualities, and their use in organizational settings could raise concerns about privacy, consent, and potential discrimination. The proposed instrument is therefore primarily intended to support individual developers in expressing and exercising their preferences.

IX. Conclusions and Future Work

Motivated by the need to preserve developers' agency over discretionary implementation choices when delegating code generation to LLM-based assistants, this paper investigated developers' discretion and the soft-ethical implications of their implementation choices. Our findings indicate that developers perceive discretion over such choices and recognize the influence of personal preferences on their decisions. We identified six relevant soft-ethical aspects and developed a 25-item scenario-based instrument to elicit developers' preferences concerning these aspects.

Future work will proceed in three directions. First, we plan to conduct a psychometric evaluation of the instrument with a larger sample of developers, following established guidelines [37]. Second, building on our proof of concept, we will investigate and validate alternative ways of representing developers' soft-ethical profiles. Finally, we will empirically evaluate whether and to what extent these profiles can guide LLM-based code generation toward implementations aligned with developers' expressed preferences.

References

[1] S. Lau and P. J. Guo, "The design space of LLM-based AI coding assistants: An analysis of 90 systems in academia and industry," in *Symposium on Visual Languages and Human-Centric Computing*, pp. 300–313, IEEE, 2025.

[2] A. Sergeyuk, Y. Golubev, T. Bryksin, and I. Ahmed, "Using ai-based coding assistants in practice: State of affairs, perceptions, and ways forward," *Information and Software Technology*, vol. 178, p. 107610, 2025.

[3] Stack Overflow, "2025 Stack Overflow Developer Survey." https://survey.stackoverflow.co/2025/, 2025. Accessed: 2026-05-20.

[4] V. Jackson, B. Vasilescu, D. Russo, P. Ralph, R. Prikladnicki, M. Izadi, S. D'angelo, S. Inman, A. Andrade, and A. Van Der Hoek, "The impact of generative ai on creativity in software development: A research agenda," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–28, 2025.

[5] D. Russo, S. Baltes, N. Van Berkel, P. Avgeriou, F. Calefato, B. Cabrero-Daniel, G. Catolino, J. Cito, N. Ernst, T. Fritz, *et al.*, "Generative ai in software engineering must be human-centered: The copenhagen manifesto," *Journal of Systems and Software*, vol. 216, p. 112115, 2024.

[6] J. T. Liang, M. Arab, M. Ko, A. J. Ko, and T. D. LaToza, "A qualitative study on the implementation design decisions of developers," in *International Conference on Software Engineering*, pp. 435–447, IEEE, 2023.

[7] L. Floridi, "Soft ethics and the governance of the digital," *Philosophy & Technology*, vol. 31, pp. 1–8, Mar 2018.

[8] R. Alidoosti, P. Lago, M. Razavian, and A. Tang, "Exploring ethical values in software systems: A systematic literature review," *Journal of Systems and Software*, vol. 226, p. 112430, 2025.

[9] M. Shahin, W. Hussain, A. Nurwidyantoro, H. Perera, R. Shams, J. Grundy, and J. Whittle, "Operationalizing human values in software engineering: A survey," *IEEE Access*, vol. 10, pp. 75269–75295, 2022.

[10] A. Nurwidyantoro, M. Shahin, M. R. Chaudron, W. Hussain, R. Shams, H. Perera, G. Oliver, and J. Whittle, "Human values in software development artefacts: A case study on issue discussions in three android applications," *Information and Software Technology*, vol. 141, p. 106731, 2022.

[11] A. Parziale, G. Voria, V. Pontillo, A. Di Salle, P. Pelliccione, G. Catolino, and F. Palomba, "Once upon a team: Investigating bias in llm-driven software team composition and task allocation," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, pp. 77–88, 2026.

[12] A. Parziale, G. Voria, V. Pontillo, G. Catolino, A. De Lucia, and F. Palomba, "Toward systematic counterfactual fairness evaluation of large language models: The caffe framework," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, pp. 3117–3129, 2026.

[13] W. Sun, J. Shi, Z. Yang, Y. Chen, H. Li, M. Yan, and D. Lo, "How reasoning shapes social bias in llm-generated code?," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*, ASE '26, (New York, NY, USA), p. 713–725, Association for Computing Machinery, 2026.

[14] L. Ling, F. Rabbi, S. Wang, and J. Yang, "Bias unveiled: Investigating social bias in llm-generated code," in *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025* (T. Walsh, J. Shah, and Z. Kolter, eds.), pp. 27491–27499, AAAI Press, 2025.

[15] Y. Liu, X. Chen, Y. Gao, Z. Su, F. Zhang, D. Zan, J. Lou, P. Chen, and T. Ho, "Uncovering and quantifying social biases in code generation," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), 2023.

[16] D. Huang, J. M. Zhang, Q. Bu, X. Xie, J. Chen, and H. Cui, "Bias testing and mitigation in llm-based code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 1, pp. 1–31, 2025.

[17] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, *et al.*, "Agent readmes: An empirical study of context files for agentic coding," *ACM Transactions on Software Engineering and Methodology*, 2025.

[18] P. Migliarini, M. Autili, P. Inverardi, and P. Pelliccione, "Ethical prompt engineering for ai-driven se: Evidence-informed interaction-time governance roadmap to 2030," *ACM Transactions on Software Engineering and Methodology*, 2026.

[19] Anonymous, "Replication package," 2026. https://anonymous.4open.science/r/developers-agency-llm-seis/.

[20] M. Petre and M. Shaw, "Contrasting to spark creativity in software development teams," *IEEE Software*, 2025.

[21] A. Caliskan-Islam, R. Harang, A. Liu, A. Narayanan, C. Voss, F. Yamaguchi, and R. Greenstadt, "De-anonymizing programmers via code stylometry," in *USENIX security symposium*, pp. 255–270, 2015.

[22] Z. Xu, C. Li, Q. Li, and S. H. Tan, "What makes code generation ethically sourced?," in *International Conference on Software Engineering*, p. 617–629, ACM, 2026.

[23] M. Autili, R. Corsi, M. De Sanctis, P. Inverardi, and P. Pelliccione, "Ethics label for digital systems to promote transparency and user awareness," *Journal of Systems and Software*, vol. 235, p. 112752, 2026.

[24] G. De Ninno, M. De Sanctis, P. Inverardi, R. Spalazzese, and C. Tsigkanos, "Developers' blind spot: Designing systems to enable stakeholders' understanding of ethical qualities," in *International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*, p. 137–141, IEEE, 2026.

[25] C. Alfieri, D. Donati, S. Gozzano, L. Greco, and M. Segala, "Ethical preferences in the digital world: The exosoul questionnaire," in *International Conference on Hybrid Human-Artificial Intelligence*, pp. 290–299, SAGE Publications, 2023.

[26] M. A. Memon, G. Filippone, G. L. Scoccia, M. Autili, and P. Inverardi, "Robethichor: Automated context-aware ethics-based negotiation for autonomous robots," *Journal of Systems and Software*, vol. 232, p. 112692, 2026.

[27] D. Donati, S. Gozzano, P. Inverardi, and N. Troquard, "Individual ethics and dispositions in the digital world," *AI & SOCIETY*, pp. 1–15, 2026.

[28] T. Santilli, M. De Luca, D. Amalfitano, A. R. Fasolino, and P. Pelliccione, "A decontextualized llm-based safeguard technique for automated jailbreak mitigation," *Information and Software Technology*, vol. 195, p. 108130, 2026.

[29] P. Ralph, N. b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, *et al.*, "Empirical standards for software engineering research," *arXiv preprint arXiv:2010.03525*, 2020.

[30] S. Baltes and P. Ralph, "Sampling in software engineering research: A critical review and guidelines," 2021.

[31] G. Voria, S. Lambiase, M. C. Schiavone, G. Catolino, and F. Palomba, "From expectation to habit: Why do software practitioners adopt fairness toolkits?," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*, pp. 94–105, IEEE, 2025.

[32] S. Lambiase, G. Catolino, F. Palomba, F. Ferrucci, and D. Russo, "Investigating the role of cultural values in adopting large language models for software engineering," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 1, pp. 1–43, 2025.

[33] P. Eyal, R. David, G. Andrew, E. Zak, and D. Ekaterina, "Data quality of platforms and panels for online behavioral research," *Behavior research methods*, pp. 1–20, 2021.

[34] B. D. Douglas, P. J. Ewell, and M. Brauer, "Data quality in online human-subjects research: Comparisons between mturk, prolific, cloudresearch, qualtrics, and sona," *Plos one*, vol. 18, no. 3, p. e0279720, 2023.

[35] D. S. Cruzes and T. Dybå, "Recommended steps for thematic synthesis in software engineering," in *International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 275–284, IEEE, 2011.

[36] J. Kontio, L. Lehtola, and J. Bragge, "Using the focus group method in software engineering: Obtaining practitioner and user experiences," in *International Symposium on Empirical Software Engineering (ISESE)*, pp. 271–280, IEEE, 2004.

[37] D. Graziotin, P. Lenberg, R. Feldt, and S. Wagner, "Psychometrics in behavioral software engineering: A methodological introduction with guidelines," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 1, 2022.

[38] Y. Wang, M. Mäntylä, S. Eldh, J. Markkula, K. Wiklund, T. Kairi, P. Raulamo-Jurvanen, and A. Haukinen, "A self-assessment instrument for assessing test automation maturity," in *Proceedings of the Evaluation and Assessment on Software Engineering (EASE)*, pp. 145–154, ACM, 2019.

[39] B. A. Kitchenham and S. L. Pfleeger, "Principles of survey research: Part 3: Constructing a survey instrument," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 2, pp. 20–24, 2002.